Exercise 1: Implementing the Singleton Pattern

Scenario

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging. The Singleton Pattern guarantees that a class has exactly one instance and provides a global point of access to it.

What is the Singleton Pattern?

The Singleton Pattern is a **Creational Design Pattern** that restricts instantiation of a class to exactly one object. It is useful when exactly one object is needed to coordinate actions across the system.

Key Characteristics	Description
Private Constructor	Prevents external classes from creating new instances
Static Instance	Holds the single instance as a private static field
Public Static Method	Provides global access to the single instance
Thread Safety	Uses synchronized keyword to prevent race conditions

Project Structure: SingletonPatternExample

The project contains two Java files:

- **Logger.java** — The Singleton class
- **SingletonTest.java** — Test class to verify the pattern

File: Logger.java

This class implements the Singleton Design Pattern for the logging utility.

```
/**
 * Logger.java - Singleton Pattern Implementation
 *
 * Ensures only ONE Logger instance exists throughout
 * the entire application lifecycle.
 */
public class Logger {

    // Step 1: Private static instance of itself (lazy initialization)
    private static Logger instance;

    // Step 2: Private constructor prevents external instantiation
    private Logger() {
        System.out.println("Logger instance created.");
    }
```

```
    // Step 3: Public static method to get the single instance (thread-safe)
    public static synchronized Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    // Logging methods
    public void log(String message) {
        System.out.println("[LOG] " + message);
    }

    public void info(String message) {
        System.out.println("[INFO] " + message);
    }

    public void warn(String message) {
        System.out.println("[WARN] " + message);
    }

    public void error(String message) {
        System.out.println("[ERROR] " + message);
    }
}
```

File: SingletonTest.java

Test class to verify that only one Logger instance is created and reused.

```
/**
 * SingletonTest.java
 * Test class to verify the Singleton Pattern implementation
 */
public class SingletonTest {

    public static void main(String[] args) {

        System.out.println("=== Singleton Pattern Test ===\n");

        // Get first instance
        Logger logger1 = Logger.getInstance();
        logger1.log("Application started");

        // Get second instance
        Logger logger2 = Logger.getInstance();
        logger2.info("User logged in");

        // Get third instance
        Logger logger3 = Logger.getInstance();
        logger3.warn("Disk space running low");
        logger3.error("Connection timeout");

        System.out.println("\n--- Verifying Singleton Behavior ---");

        // Verify all references point to the same instance
```

```
        System.out.println("logger1 == logger2 : " + (logger1 == logger2));
        System.out.println("logger2 == logger3 : " + (logger2 == logger3));
        System.out.println("logger1 == logger3 : " + (logger1 == logger3));

        System.out.println("\nHash Codes:");
        System.out.println("logger1 hashCode: " + logger1.hashCode());
        System.out.println("logger2 hashCode: " + logger2.hashCode());
        System.out.println("logger3 hashCode: " + logger3.hashCode());

        if (logger1 == logger2 && logger2 == logger3) {
            System.out.println("\n SUCCESS: All references point to the SAME Logger instance.");
            System.out.println("  Singleton Pattern is correctly implemented!");
        } else {
            System.out.println("\n FAILURE: Multiple Logger instances detected!");
        }
    }
}
```

Expected Output

```
=== Singleton Pattern Test ===

Logger instance created.
[LOG] Application started
[INFO] User logged in
[WARN] Disk space running low
[ERROR] Connection timeout

--- Verifying Singleton Behavior ---
logger1 == logger2 : true
logger2 == logger3 : true
logger1 == logger3 : true

Hash Codes:
logger1 hashCode: 1829164700
logger2 hashCode: 1829164700
logger3 hashCode: 1829164700

 SUCCESS: All references point to the SAME Logger instance.
  Singleton Pattern is correctly implemented!
```

How it Works — Step by Step

Step 1 — Private Static Field

private static Logger instance; declares a static field to hold the single Logger object. Being static means it belongs to the class, not any particular instance.

Step 2 — Private Constructor

private Logger() {} makes the constructor private so no external class can call new Logger(). This is the key restriction that enforces the Singleton.

Step 3 — Public Static Accessor

getInstance() checks if instance is null. If yes, it creates one (lazy initialization). If not null, it returns the existing instance. The synchronized keyword ensures thread safety.

Step 4 — Verification

All three references (logger1, logger2, logger3) point to the identical object in memory. This is confirmed by == comparison and identical hashCode values.

Advantages of the Singleton Pattern

- Controlled Access: Only one instance is created, reducing memory overhead.
- Global State: Provides a single point of access across the entire application.
- Lazy Initialization: Instance is created only when first requested.
- Thread Safety: The synchronized getInstance() method prevents race conditions.
- Consistent Logging: All parts of the application share the same logger state.